WHITEPAPER · OCTOBER 2026

Duck Bank

Open borrowing in RLUSD on the XRP Ledger.
One borrower per vault. Lenders hold the vault MPT share.
Beside each loan, the borrower's XRP sits in an escrow to the broker.



QAK: 1,000,000,000 supply, issued on FirstLedger in Q4 2026. Issuer blackholed. Issuer half in two escrows.

Lending: built on XLS-65 (Single Asset Vault) and XLS-66 (Lending Protocol). Mainnet deposits open only after SingleAssetVault, LendingProtocol and LendingProtocolV1_1 are enabled on mainnet. On 8 October 2026 none of them is.

1. The desk

Duck Bank lends RLUSD to any account that has paid the one-time access fee and posts XRP collateral: an XRP escrow held beside the loan. Each borrower gets its own closed-ended vault, its own loan broker and one loan. Lenders choose a borrower, deposit during the subscription window and receive **the vault MPT share**, the receipt for that borrower only. If the borrower misses, the loss is written down on that vault. The next borrower is a different vault. There is no vote.

QAK is the app token. A one-time payment of 1,000,000 QAK gives an account lifetime access (section 6). This is a rule of the Duck Bank app. The ledger knows nothing about it, and no QAK ever stands as loan cover.

Before LoanSet the borrower locks XRP in an escrow to the broker wallet (section 5). XLS-66 does not see that escrow. The broker's bot finishes and sells it if the borrower misses, which is an app step, not a ledger step. Without an escrow there is no LoanSet.

Why one vault per borrower. A shared vault with many loans would spread one borrower's write-down across every lender in it. One vault per borrower keeps each loss on the lenders who chose that borrower, and lets `DebtMaximum`, the cover and the 150/120 lines apply to one loan.

2. The ledger model

Every field, flag and error below is taken from the XLS-65 and XLS-66 specifications (including the closed-ended vault extension and LendingProtocolV1_1).

#	Transaction	What Duck Bank sets
1	`VaultCreate`	Asset RLUSD; `VaultKind` 1 (ClosedEnded) with `SubscriptionDate` and `RedemptionDate`; `AssetsMaximum` = the cap; `Scale` 6 (IOU range 0–18); flag `tfVaultShareNonTransferable`. The optional borrower label stays off-ledger.
2	`LoanBrokerSet`	On that VaultID: `DebtMaximum` = the cap, `CoverRateMinimum` 15000 (15%), `CoverRateLiquidation` 100000 (100%), `ManagementFeeRate` 1000, 1% of interest (fixed once the broker exists).
3	`LoanBrokerCoverDeposit`	RLUSD first-loss cover. Only the broker owner can deposit, and only the vault asset (`tecWRONG_ASSET` otherwise).
4	`VaultDeposit`	Only while now ≤ SubscriptionDate; later deposits fail `tecEXPIRED`. Withdrawals fail `tecTOO_SOON` until RedemptionDate.
5	`LoanSet`	Only in the investment window. Signed by the broker and by the borrower (`CounterpartySignature`; without it `temBAD_SIGNER`). StartDate + PaymentInterval × PaymentTotal + 60 s must be ≤ RedemptionDate. `GracePeriod` ≤ `PaymentInterval`.
6	`LoanPay`	Fixed installments, sent only by the borrower (anyone else gets `tecNO_PERMISSION`). A payment below the periodic payment fails `tecINSUFFICIENT_PAYMENT`; a late one needs `tfLoanLatePayment` and settles exactly one installment. On an impaired loan LoanPay works and lifts the impairment; after default it fails `tecKILLED`.
7	`LoanManage`	`tfLoanImpair` once a payment is overdue; `tfLoanDefault` once now > NextPaymentDueDate + GracePeriod.
8	`LoanDelete`	When PaymentRemaining is 0: the loan is settled or written off.

One vault serves one borrower, and never a second. The vault's pseudo-account holds the RLUSD and issues the shares. It cannot receive ordinary payments, so money reaches lenders only through `VaultDeposit`, `LoanPay` and, on default, cover moved by `LoanManage`.

> The lender holds the vault MPT share. It is non-transferable, redeemable with `VaultWithdraw` from RedemptionDate, and it tracks that one vault's assets, including any loss written down on it.

3. The first pool

One sample borrower, a 10,000 RLUSD cap, short term. Terms: `InterestRate` 10000 (10% a year), `PaymentTotal` 3, `PaymentInterval` 30 days, `GracePeriod` 7 days. The vault and broker have not been created yet, so there is no VaultID or LoanBrokerID to publish.

Maximum principal under the cap

`AssetsMaximum` = `DebtMaximum` = 10,000 RLUSD. XLS-66 counts the expected interest toward both limits. LoanSet fails if AssetsTotal ≥ AssetsMaximum, if AssetsTotal + InterestDue > AssetsMaximum, or if DebtTotal + Principal + InterestDue > DebtMaximum (all `tecLIMIT_EXCEEDED`). A full 10,000 deposit plus a 10,000 principal therefore cannot be lent.

Item	Value	How
Periodic payment	3,333.73 × 3	Amortized payment, periodic rate = 10% × 30 days / 365 days (XLS-66 A-2)
Maximum principal	**9,839.42 RLUSD**	Largest principal with principal + InterestDue ≤ 10,000, rounded down to the cent with a 0.01 margin
InterestDue to the vault	160.56 RLUSD	Gross interest less the management fee, at fee rate 1000
Principal + InterestDue	9,999.98 RLUSD	≤ AssetsMaximum and ≤ DebtMaximum
Deposit target	9,839.42 RLUSD	The app stops inviting deposits at the principal
Cover before LoanSet	1,500.00 RLUSD	(DebtTotal + Principal + InterestDue) × CoverRateMinimum 15%

The vault is public. The ledger itself accepts deposits up to AssetsMaximum, so direct deposits can push AssetsTotal to 10,000. In that case LoanSet fails and lenders wait for RedemptionDate to withdraw.

4. When a borrower misses

There is no on-chain collateral and no liquidation. The XLS-66 security section states that the protocol does not offer on-chain algorithmic protection against default. What the ledger does, after the grace period, is `LoanManage` with `tfLoanDefault`:

> DefaultCovered = min(DebtTotal × CoverRateMinimum × CoverRateLiquidation, DefaultAmount, CoverAvailable)
> Vault loss = DefaultAmount − DefaultCovered, written down on that vault only.

Worked on the first pool, if nothing is repaid: DefaultCovered = min(9,999.98 × 15% × 100%, 9,999.98, 1,500.00) = 1,500.00 RLUSD of cover moved into the vault, and 8,499.99 RLUSD written down on that vault. No other vault is touched. The XLS-66 text books expected interest into AssetsTotal at LoanSet; on devnet with LendingProtocolV1_1 the ledger booked interest only as payments arrived (seen in our devnet run). The app counts expected interest toward the caps and the cover, which is the stricter bound. Redemption still opens on RedemptionDate even if the loan is late.

The borrower's XRP escrow (section 5) does not change this formula. Whatever the bot's sale brings in reaches the vault only as a LoanPay that the borrower has signed, before the default; the rest is still written down.

5. The XRP escrow beside the loan

Escrow fields and errors are from the XRPL Escrow documentation; the swap math is from the AMM (XLS-30) and DEX documentation. XLS-66 does not take this escrow as collateral, and it never sells it. Everything the ledger does not enforce is marked **app rule**.

#	Step	Who	What the ledger checks
1	`EscrowCreate` before LoanSet	borrower (Xaman)	XRP `Amount` in drops to `Destination` = the broker wallet; `Condition` PREIMAGE-SHA-256 (the only condition type XRPL supports); `CancelAfter` required for a conditional escrow; optional `FinishAfter` before it. The broker makes the preimage and keeps the fulfillment off the app.
2	Find the escrow	app	`ledger_entry` by owner + sequence: Destination, Condition, Amount and CancelAfter must match. LoanSet stays blocked in the app until this passes, and until the opening ratio is at least 150% (app rules).
3	`LoanSet`	broker + borrower	As in section 2. The escrow is not a LoanSet field.
4	Watch the ratio	app (read-only)	`amm_info` and `book_offers` on XRP/RLUSD (mainnet read; XRP/DUSD on devnet, or "no market").

> Ratio = what the escrowed XRP would fetch now if sold (AMM formula, order-book walk, or both, whichever is best) ÷ RLUSD still owed after cover. "Owed after cover" = DefaultAmount − DefaultCovered from the ledger's own default formula, so only the cover the ledger would actually move counts.
> **Add line 150%** · **default line 120%** (app rules; both shown live on the pool page).

Under 150%, the borrower has a window (config: 72 hours) to add a **second escrow** (an existing escrow cannot be increased: the only escrow transactions are EscrowCreate, EscrowFinish and EscrowCancel) or to LoanPay the balance down. If the window ends under 120%, the bot runs. On the first pool's planned figures (8,499.98 RLUSD owed after 1,500.00 of cover), 150% needed about 9,010 XRP on mainnet XRP/RLUSD at 13:00 EEST on 8 October 2026 (validated ledger 107514201), and 120% about 7,207 XRP. This moves with the market.

The four sale steps (each shown as done or stuck)

Step	Transaction	Notes
1. Mark	`LoanManage` tfLoanImpair	The ledger allows it only once a payment is overdue. A price breach alone, with payments current, waits for the next due date.
2. Finish	`EscrowFinish` + fulfillment	Only after FinishAfter and before CancelAfter. Any account may submit it; the XRP goes to the broker.
3. Sell	`OfferCreate` tfFillOrKill + tfSell	Fills against the AMM and the order book, with a 3% limit below the quote (config). Fill-or-kill: either all the XRP is sold at or above the limit, or nothing is. Nothing rests on the book.
2+3 together	one XLS-56 `Batch`, ALLORNOTHING	Steps 2 and 3 and the Payment of the LoanPay amount to the borrower go out as one Batch (amendment BatchV1_1): every EscrowFinish, the OfferCreate and the Payment apply together, or none of them do. So the sell leg is atomic. The bot checks each inner transaction by hash on the validated ledger, because the outer Batch returns tesSUCCESS even when inner transactions fail. A Batch holds at most 8 inner transactions; escrows beyond that are finished first and their XRP is sold in the Batch.
4. LoanPay	the borrower's presigned `LoanPay`	LoanPay is borrower-only, so the borrower signs it in Xaman without submitting. The Batch sends the borrower exactly that amount, and the bot then submits the borrower's LoanPay. The output of a sale is not known in advance, and a Batch carries fixed amounts. The Payment is therefore included only when the LoanPay amount is at most the fill-or-kill minimum (quote − 3%). Any output above it stays with the broker as leftover for the lenders. If the minimum is below the LoanPay amount, the Payment and the LoanPay follow the Batch as separate transactions, and only if the actual output covers them. A vault deposit is not a route: VaultDeposit after SubscriptionDate fails `tecEXPIRED`, and the pseudo-account cannot receive payments.

Then `LoanManage` tfLoanDefault for the remainder, once the ledger allows it. Any shortfall is written down on that vault. The pool page derives all four steps from ledger reads (Loan flags, escrow entries, validated transactions); the browser holds no keys. The bot is a broker-side Node script.

What Batch and delegation cannot do here

- **LoanPay stays presigned.** Loan* and Vault* transactions cannot be Batch inner transactions: the xrpl.org Batch reference lists LoanSet, LoanPay, LoanManage, LoanDelete, every LoanBroker* and every Vault* type as disabled (`temINVALID_INNER_BATCH`). They cannot be delegated either: XLS-75 lists the same types as not delegable, and a DelegateSet granting LoanPay fails `temMALFORMED` (devnet probe, 12 October 2026). So impair and default stay separate LoanManage transactions, and the borrower's LoanPay stays a presigned transaction that the bot submits.
- **Escrow + LoanSet atomicity waits.** The borrower's EscrowCreate and the LoanSet cannot be one Batch until a future LendingProtocolV1_2 amendment lifts the inner-transaction ban on Loan* (it is not yet supported by any xrpld release). Until then the app gates LoanSet on the escrows already being on the ledger.
- **Payment delegation is deliberately not used.** A delegated Payment permission cannot be limited to an amount, a currency or a destination. Granting it to the bot would let the bot spend all of the borrower's RLUSD. The presigned LoanPay is the narrower authority.
- **Bot keys.** The bot runs from a hot account that submits the Batch and the EscrowFinish itself (any account may finish a condition escrow; the XRP still goes to the broker). The broker key stays cold and signs only LoanManage, LoanBrokerSet and cover. The broker grants the hot account `OfferCreate` only (XLS-75 DelegateSet); a delegated Payment or EscrowCreate fails `terNO_DELEGATE_PERMISSION` (devnet probe, 12 October 2026). The funding Payment to the borrower and the leftover payments need a separately approved payer key, because a Payment grant cannot be limited; if that key is the broker key and kept online, the separation gives nothing for that leg.

- **The borrower can void its presigned LoanPay.** After the funding Payment arrives, the borrower can reuse the LoanPay's Sequence or Ticket, or move the funds away (devnet: `tefPAST_SEQ`, `tefNO_TICKET`, `tecINSUFFICIENT_FUNDS`). No XRPL primitive ties a payment to a later LoanPay; a Check or token escrow would also hand the funds to the borrower. The bot prefers a Ticket-based LoanPay, checks that the blob's Sequence or Ticket is still live before funding, tags the funding with a `duckbank/fund` memo and reads it back from the ledger, uses a tight LastLedgerSequence, submits the LoanPay right after the funding validates, and funds once only: a voided blob stops all further funding to that borrower and the amount is recorded as owed by the borrower. Residual risk: a racing borrower can keep one funding, at most one installment, per loan.

Leftover proceeds go to the lenders, pro rata

> App rule the broker promises. The ledger does not enforce it, and it sits outside the vault and the loan.

Sale proceeds that the bot could not put into the loan by LoanPay (less than one installment, or anything still with the broker once the loan is defaulted or closed) are paid by the broker wallet straight to that pool's lenders, as a `Payment` in the vault asset (RLUSD; DUSD on devnet).

Choice	Rule
Basis	Each lender's vault MPT share balance at the snapshot: the ledger of the `LoanManage` default (up to that transaction), or of the LoanPay that closed the loan. Read-only: Clio `mpt_holders` for the vault's ShareMPTID at that ledger; where the server is not Clio, the balances are rebuilt from the MPToken entries in the vault's VaultDeposit/VaultWithdraw history, so lenders who have since withdrawn still count.
Who counts	Every share holder. The vault pseudo-account issues the shares and holds none. Shares the broker holds count like any lender's; that part needs no payment.
Rounding and dust	Amounts are floored to 0.000001. The remainder (fewer micro-units than there are holders) goes to the largest holder; ties go to the lowest address. Below 0.000001 in total, nothing is paid.
No trust line	A Payment cannot deliver the asset to an account without a trust line for it. That lender's amount stays recorded as owed and is shown on the pool page.
Proof of payment	Each Payment carries the memo `duckbank/leftover` with the LoanID, so the pool page lists every lender's amount with its payment hash and Batch hash, or the amount still owed.
Sending	Up to 8 Payments per XLS-56 Batch, in INDEPENDENT mode: one lender who cannot receive does not block the others, and that amount stays owed. Before each Batch the bot reads the memo-tagged payments already on the ledger and pays only what is still owed, so a retry never pays a lender twice.

Where the ledger and the plan differ

Point	Ledger fact	How Duck Bank handles it
Finish timing	The broker can finish whenever it holds the fulfillment, after FinishAfter and before CancelAfter.	"A price drop never finishes the escrow" and "the sale comes only after LoanManage" are app rules (broker promises). FinishAfter = SubscriptionDate + PaymentInterval, the first moment any payment can be overdue, so the ledger does block an earlier finish.
XRP back to a repaying borrower	Only `EscrowCancel` after CancelAfter (before it: `tecNO_PERMISSION`, seen on devnet).	CancelAfter = RedemptionDate + the loan's GracePeriod + a 24-hour buffer. The app's LoanSet deadline is RedemptionDate − term − grace − sale window, so the last due date, the grace period and the bot's sale window all end before CancelAfter; the grace period and the buffer leave room for the bot's EscrowFinish and sale if the sale runs late, since the ledger refuses EscrowFinish after CancelAfter. A borrower that repays waits for CancelAfter, unless the broker finishes and sends the XRP back earlier (app rule).
Getting proceeds into the vault	LoanPay is borrower-only; it works on an impaired loan and fails `tecKILLED` after default. A late LoanPay settles one installment, with no partial payments.	Order: impair → finish → sell → the borrower's LoanPay while impaired → default the remainder. Proceeds below one installment cannot go into the loan, so the vault books that part as shortfall at default. Without the borrower's signature, step 4 is stuck. Those leftover proceeds then go to the pool's lenders, pro rata (below).
Timing in config	Default needs now > NextPaymentDueDate + GracePeriod.	Sale window 3 days (config) after the grace period; investment window = term + grace + sale window + 3 days of slack (first pool: 103 days).

No cover-only path

A borrower without an XRP escrow cannot get a LoanSet. The earlier cover-only path is removed.

6. QAK: lifetime access

Rule	What it does
Fee	One QAK `Payment` of 1,000,000 QAK (issuer `r98RkKUasH5vA3mshD5Bi3eAVHCYwziY9M`) from the borrower's account to `rpRVzQSeZxxakWryRuTchxGQN7cs5oPi52` gives that account lifetime access to apply. Paid once.
Check	The app reads the borrower's mainnet `account_tx`, page by page, and accepts a validated `tesSUCCESS` Payment to that address with `delivered_amount` of at least 1,000,000 QAK of that issuer. A partial payment does not count. Access belongs to the paying account only.
Pay	The app builds a Xaman payload pinned to the connected account, memo `duckbank/access`. Mainnet signing is off in this build, so the payment is made from another wallet until mainnet opens; the check reads the ledger either way.

> Access is not a vote, not cover and not collateral. The QAK paid is not `CoverAvailable`: cover accepts only the vault asset, RLUSD. On default the ledger takes RLUSD cover only. QAK is never escrowed for a loan and never sold by the bot; the borrower's escrow holds XRP.

7. Tokenomics

QAK was created on FirstLedger in Q4 2026. Supply is 1,000,000,000, fixed.

Bucket	Of supply	Tokens	Use
Public pool	50%	500,000,000	Sold into the opening FirstLedger pool.
Team	10%	100,000,000	McQAK, Donald QAK, Della and Huey QAK. From the escrowed issuer half.
Marketing	20%	200,000,000	Onboarding, content and the desk. From the escrowed issuer half.
AMM	20%	200,000,000	Added to the QAK/XRP AMM after unlock. From the escrowed issuer half.

Launch

FirstLedger, 50 XRP starting liquidity, 1% AMM fee, 15-minute anti-sniper.

Issuer and escrows (read live from the ledger on 8 October 2026)

The issuer `r98RkKUasH5vA3mshD5Bi3eAVHCYwziY9M` is blackholed: master key disabled, regular key set to the blackhole address `rrrrrrrrrrrrrrrrrrrrBZbvji`, no signer list. No more QAK can be issued. The issuer half, 500,000,000 QAK, is in two escrows:

Escrow	Tokens	Unlocks (FinishAfter)
1	125,000,000 QAK	8 October 2026, 15:49 EEST
2	375,000,000 QAK	7 November 2026, 16:24 EET

8. Status and gating

XLS-65/66 need three amendments: SingleAssetVault, LendingProtocol and LendingProtocolV1_1. A rippled release is not activation: an amendment counts only once validators enable it on that network. The app's Status page runs the read-only `feature` request on each network. On 8 October 2026: mainnet none enabled, testnet none enabled, devnet all three enabled.

- **Mainnet:** vault lending is off in the build itself, regardless of amendment status. No mainnet deposits.
- **Testnet:** status only.
- **Devnet:** the real transactions, once all three amendments read enabled. RLUSD does not exist on devnet, so the app uses a labelled test IOU from a configured test issuer; with none configured, devnet actions stay disabled.

Devnet run of the escrow path (8 October 2026, 13:13–13:35 EEST, DUSD test token, cap 50). The borrower escrowed 15 + 9 XRP to the broker wallet. The app kept LoanSet blocked at a 127.6% ratio and opened it at 176%. After a price drop and a missed installment, the bot marked the loan, finished both escrows, sold 24 XRP for 31.53 DUSD and paid 24.99 DUSD as the borrower's own signed LoanPay. LoanPay worked on the impaired loan; nothing was blocked. The remaining 6.55 DUSD was below one installment. After the next miss, the default moved 3.75 DUSD of cover, and 21.24 DUSD was written down on that vault. The broker's LoanPay (`tecNO_PERMISSION`), the borrower's early EscrowCancel (`tecNO_PERMISSION`), a VaultDeposit after SubscriptionDate (`tecEXPIRED`) and LoanPay after default (`tecKILLED`) were rejected as expected; after CancelAfter, a third party cancelled leftover escrows and the XRP went back to the borrower. The leftover 6.547539 DUSD was then paid by the bot's `distribute` command to the only share holder at the default (100% of the shares; the snapshot was rebuilt from the vault's history, as devnet has no Clio `mpt_holders`). Every hash is on the app's Status page.

Devnet run of the batched path (12 October 2026, 09:55–10:11 EEST, DUSD test token, cap 50, two lenders). Two lenders deposited 30 and 19.98 DUSD and the borrower escrowed 13 + 7 XRP. After a price drop and a missed installment the bot impaired the loan, then sent one ALLORNOTHING Batch (`A293659F…2F35CA`): both EscrowFinish, an OfferCreate fill-or-kill selling 20 XRP with a 30.601854 DUSD minimum (it got 31.548304), and a Payment of 24.985103 DUSD to the borrower. The borrower’s presigned LoanPay of 24.985103 followed. The sale paid part of the debt; after the second miss the bot defaulted the rest (3.747755 DUSD of cover moved, 21.237279 DUSD written down on that vault). The 6.563201 DUSD left over went out in one INDEPENDENT Batch (`27CD3F82…C40A462`): 3.939496 to lender 1 and 2.623704 to lender 2, pro rata to their shares at the default. A second distribute run paid nobody. Every hash is on the Status page.

Xaman is the wallet. Its network list includes DEVNET, so sign requests can be forced to devnet. Xaman support for the XLS-65/66 transaction types is not verified, and Xaman does not document producing a LoanSet `CounterpartySignature`, so the borrower's counter-signature cannot yet be collected in the app. EscrowCreate is a standard mainnet transaction type, and Xaman's developer documentation says it accepts all mainnet transaction types. Without an API key, the app runs a wallet labelled mock that signs nothing.

9. Risks

- **Clawback.** The RLUSD issuer has clawback enabled. It can claw back RLUSD from a vault and broker cover above the minimum.
- **No on-chain collateral.** Loans are unsecured on the ledger. The borrower's XRP escrow sits beside the loan; XLS-66 does not take it as collateral.
- **No ledger liquidation.** On default only RLUSD cover, up to the rates, reaches the vault. The bot's XRP sale is an app step: it may fall short (thin book, price gap, the 3% limit), it needs the borrower's signed LoanPay, and the broker can finish an escrow at any time it holds the fulfillment. The rest is written down on that vault.
- **Batch.** Earlier Batch versions were withdrawn for security bugs before BatchV1_1 replaced them. Duck Bank uses Batch only in the devnet bot; the Status page shows each network's amendment status from the live check, and these two amendments are not gates. An outer tesSUCCESS hides inner failures, so the bot checks every inner transaction. A fill-or-kill sale can be killed by a moving price; the Batch then applies nothing and is retried with a fresh quote.
- **Leftover to lenders is a broker promise.** The ledger does not enforce it; a lender with no trust line for the asset cannot receive it, and that amount stays recorded as owed.

- **Lock-up.** Withdrawals are blocked from SubscriptionDate to RedemptionDate.
- **Interest booking.** The XLS-66 text books expected interest at LoanSet; devnet booked it as paid. The app counts expected interest toward the caps.
- **XRP locked.** A repaying borrower's XRP stays in escrow until CancelAfter. If LoanSet never happens, it also stays until CancelAfter.
- **Software.** The code is not audited.

10. Team

The 10% team bucket covers four published roles.

McQAK, Founder
The desk: the caps, the terms and the access rule.

Donald QAK, CTO
The app, the Xaman connection, the vault path and the escrow bot.

Della, Marketing
The channel, borrower onboarding and the public note when a pool opens or repays.

Huey QAK, Developer
The pool page, the access check and the test signatures.

11. Roadmap

When	Step	What
Q4 2026	Mint (done)	QAK issued on FirstLedger: 1,000,000,000 supply, 50 XRP starting liquidity, 1% AMM fee, 15-minute anti-sniper. Issuer blackholed; issuer half in two escrows.
Next	Devnet vault, broker and XRP escrow bot	One borrower on devnet with the DUSD test token: closed-ended vault, loan broker, cover, deposits, the borrower's XRP escrow to the broker, a dual-signed LoanSet, LoanPay, and the broker-side escrow bot (LoanManage, EscrowFinish, sell, LoanPay).
After the three amendments are enabled on mainnet	Mainnet deposits	Mainnet deposits open only once SingleAssetVault, LendingProtocol and LendingProtocolV1_1 are all enabled on mainnet.

12. References

- XLS-65 Single Asset Vault, including the closed-ended vault extension (XLS-65.1.4).
- XLS-66 Lending Protocol, including LendingProtocolV1_1.
- XRPL documentation: Escrow, EscrowCreate, EscrowFinish, EscrowCancel; crypto-conditions; Automated Market Makers (XLS-30), amm_info; book_offers; OfferCreate.
- App: https://trustline-tan.vercel.app (Status page: live amendment check). Venue: FirstLedger. Wallet: Xaman.

This paper describes software and token rules. It is not an offer of securities or a promise of returns. Ledger facts were read on 8 October 2026.